

IT3071 – Machine Learning and Optimization Methods

Perceptron Convergence Theorem

H.M. Samadhi Chathuranga Rathnayake

Understanding the Perceptron

- A perceptron is a type of artificial neural network invented in 1958 by Frank Rosenblatt. It is the simplest form of a neural network, used for binary classification tasks.
- Here are the key components and concepts related to a perceptron:
 - Input Nodes: These are the features of the input data. Each input node is assigned a weight.
 - Weights: Each input is multiplied by a weight which can be adjusted during the learning process to minimize error.
 - Summation: The weighted inputs are summed together.
 - Activation Function: The sum of the weighted inputs is passed through an activation function. In the case of a perceptron, a step function is often used. The output of this function is the prediction of the perceptron.
 - Bias: An additional parameter that allows the activation function to be shifted to the left or right, improving the model's fit.
- A perceptron is build around a non-linear neuron, namely McCulloch-Pits model of a neuron.

Neuronal Model with Linear Combiner and Hard Limiter

- Neuronal model with a linear combiner and hard limiter shares foundational concepts with the perceptron model, especially in terms of its activation mechanism and binary classification capability.
- Understanding these connections helps in grasping how neural network theory, works.

Components of the Neuronal Model are

- Linear Combiner
- Induced Local Field (Axon Hillock)
- Hard Limiter

Linear Combiner

- The linear combiner sums up the inputs received from various synapses, which are weighted connections between neurons.
- Additionally, it incorporates an externally applied bias to adjust the sensitivity of the model.

Induced Local Field

- This resulting sum, also referred to as the induced local field, represents the total input to the neuron after weighting and bias adjustments.

Hard Limiter

- The induced local field is then passed through a hard limiter function.
- If the induced local field is positive, the output of the hard limiter is +1.
- If the induced local field is negative, the output is -1.

Classification Goal

The primary objective of this neuronal model is to classify inputs $x_1, x_2, \dots, x_m$ into one of two classes, C1 or C2:

- **Class C1:** Inputs are classified as C1 if the output of the neuronal model is +1 (indicating a positive induced local field).
- **Class C2:** Inputs are classified as C2 if the output of the neuronal model is -1 (indicating a negative induced local field).

Classification Goal

- The Neuronal Model with Linear Combiner and Hard Limiter encapsulates a basic form of a perceptron, where the hard limiter function serves as a simple activation function.
- The Perceptron Convergence Theorem assures convergence if the data is linearly separable, demonstrating the algorithm's effectiveness in learning linear decision boundaries.

Understanding Perceptron Convergence Theorem

- The Perceptron Convergence Theorem is a fundamental result in the theory of neural networks.
- This theorem provides a guarantee about the performance of the perceptron algorithm under certain conditions.

Perceptron Convergence Theorem

- *The Perceptron Convergence Theorem states that if there exists a linear separation (a hyperplane) that can perfectly classify a given set of training examples, then the perceptron algorithm will converge to a solution that correctly classifies all the training examples.*

Conditions for Convergence

For the perceptron algorithm to converge, the following conditions must be met:

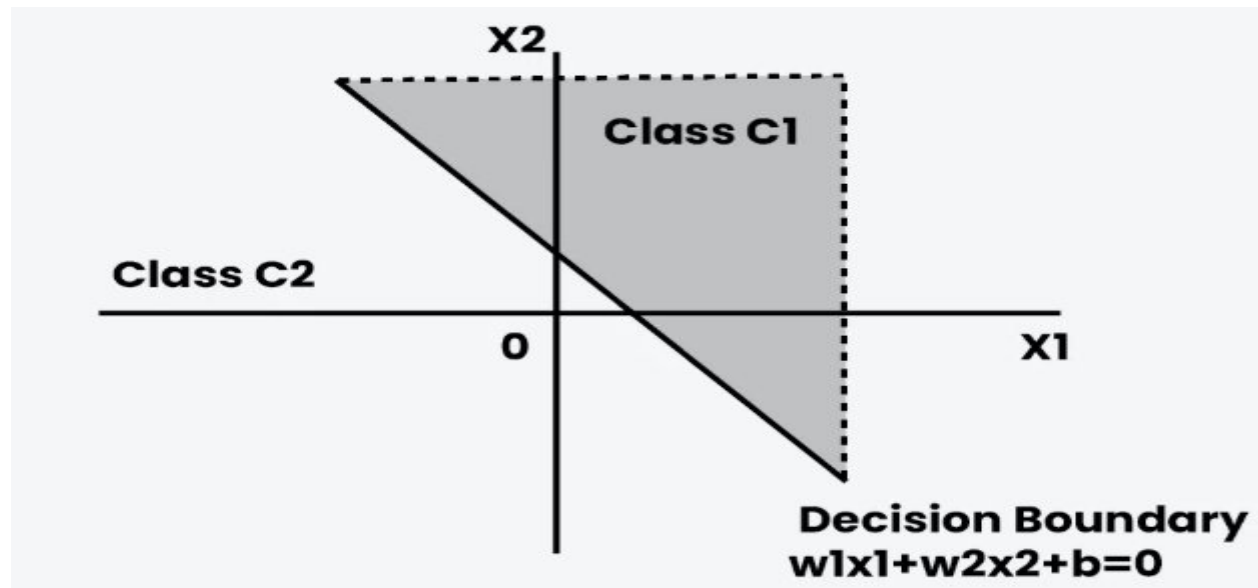
1. **Linear Separability:** The training data must be linearly separable. This means there exists a hyperplane that can separate the positive and negative examples without any errors.
2. **Non-zero Margin:** There must be a positive margin γ such that for all training examples (x_i, y_i) , the following condition holds:

$$y_i(w \cdot x_i + b) \geq \gamma$$

where w is the weight vector and b is the bias.

Implications for Theorem

- Convergence: For linearly separable data, the perceptron algorithm will always find a hyperplane that correctly classifies the training examples.
- For example- classes C1 and C2 can be classified by a hyperplane defined as:

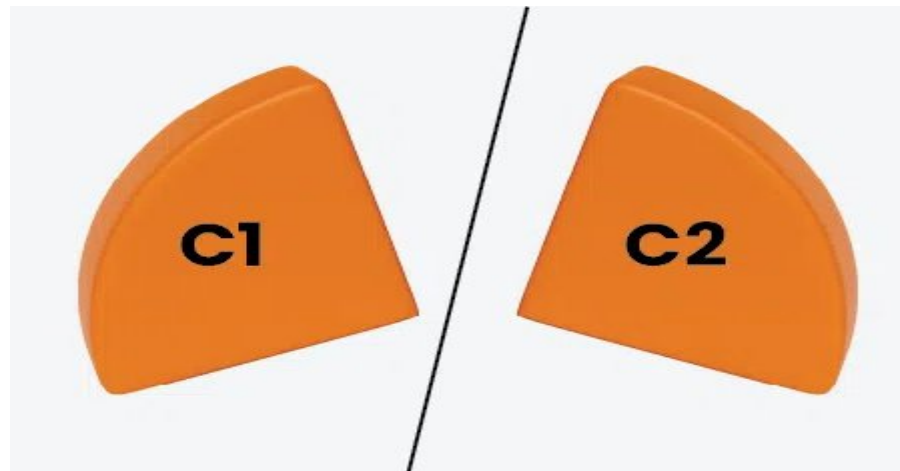


Implications for Theorem

- Separable Data: If the data is not linearly separable, the perceptron algorithm will not converge and will continue to update its weights indefinitely.
 1. Linearly Separable Data
 2. Non-linearly Separable Data

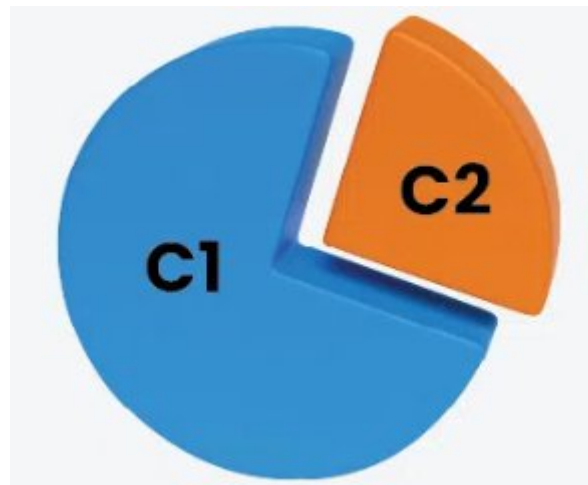
Linearly Separable Data

- It refers to a set of data points can be perfectly divided into two distinct classes using a straight line (in two dimensions), a plane (in three dimensions), or a hyperplane (in higher dimensions).
- This means that there exists a linear boundary that can clearly separate all the data points of one class from those of the other class without any overlap.



Non-linearly Separable Data

- It refers to a set of data points cannot be divided into two distinct classes using a straight line (in two dimensions), a plane (in three dimensions), or a hyperplane (in higher dimensions).
- In other words, no single linear boundary can perfectly separate the two classes.



Mathematical Formulation

- The simple neuronal model is modified for perceptron convergence theorem.
- In order to do that, bias is treated as a synaptic weight driven by a fixed input which is equal to +1.

Mathematical Formulation

- **Now input vector is defined as:**
 - $x(n) = [+1, x_1(n), x_2(n), \dots, x_m(n)]$ where n is the iteration step of the algorithm.
- **weight vector** is defined as:
 - $w(n) = [b(n), w_1(n), w_2(n), \dots, w_m(n)]$
- **Linear combiner output** $v(n)$ can be calculated as:
 - $v(n) = \sum_{i=1}^m w_i(n)x_i(n) = w^T(n)x(n)$

For a fixed n , $w^T x = 0$ plotted in m -dimensional space defines a hyperplane as decision surface between two different classes.

Mathematical Formulation

- Training:
 - Let $L1$ be the subset of training vectors which belongs to class $C1$ and $L2$ be the subset of training vector which belongs to class $C2$.
- Classifier Training:
 - $L1$ and $L2$ will train the classifier if we adjust the weight vector w such that classes $C1$ and $C2$ are linearly separable:

$$w^T x > 0 \text{ for } x \in C1$$

$$w^T x \leq 0 \text{ for } x \in C2$$

Mathematical Formulation

Algorithm for updating weight vector is:

1. If n^{th} member $x(n)$ is correctly classified by $w(n)$ at the n^{th} iteration, then no correction is made. $w(n+1) = w(n)$
2. Otherwise, weight is updated as:
 - $w(n+1) = w(n) - \eta x(n)$, if $w^T(n)x(n) > 0$ and $x(n) \in C2$
 - $w(n+1) = w(n) + \eta x(n)$, if $w^T(n)x(n) \leq 0$ and $x(n) \in C1$

If η is constant, then we have a fixed increment adaptation rule.

If η is variable, then:

$$\eta(n)x^T(n)x(n) > |w^T(n)x(n)|$$

Workings of Perceptron Convergence Algorithm

1. **Initialization:** Start by setting the weights and bias to small random values. These weights determine the importance of each input feature.
2. **Input:** Feed an input data point (a vector of features) into the perceptron.
3. **Weighted Sum:** Calculate the weighted sum of the input features plus the bias. This is done by multiplying each input feature by its corresponding weight and adding them all up, along with the bias.

$$\text{weighted sum} = (w_1 * x_1) + (w_2 * x_2) + \dots + (w_n * x_n) + b$$

Workings of Perceptron Convergence Algorithm

- 4. Activation Function:** Apply the activation function to the weighted sum to get the output. In a simple perceptron, this is usually a step function that outputs 1 if the weighted sum is positive, and -1 (or 0) if it's negative.
- 5. Prediction:** Compare the perceptron's output (prediction) to the actual label of the input data point (either 1 or -1).
- 6. Update Weights:** If the prediction is incorrect, adjust the weights and bias. The weights are updated to reduce the error, moving the decision boundary closer to correctly classifying the input.

Workings of Perceptron Convergence Algorithm

- The update rule is:

$$w_i = w_i + \Delta w_i$$

$$b = b + \Delta b$$

where

$$\Delta w_i = \eta * (y - y') * x_i$$

$$\Delta b = \eta * (y - y')$$

η is learning rate parameter

y is the actual label

y' is the predicted label

Workings of Perceptron Convergence Algorithm

7. **Repeat:** Repeat steps 2-6 for all data points in the training set. This process is called an epoch. Multiple epochs are performed until the perceptron correctly classifies all training data points or a maximum number of epochs is reached.
8. **Convergence:** If the data is linearly separable, the algorithm will eventually find the correct weights and bias to perfectly classify the training data. This means the perceptron has "converged."

Advantages of Perceptron Convergence Theorem in Neural Networks

- **Simple to implement-** It is simple and intuitive algorithm to implement especially for binary classification problems.
- **Guaranteed convergence-** It is guaranteed to converge to a linear separator if the data is linearly separable.
- **Fast Convergence-** It can converge quickly, especially when the data is well-separated.

Limitations of Perceptron Convergence Theorem in Neural Networks

- **Only suitable for linearly separable data-** This algorithm works if the data can be separated by a linear hyperplane.
- **Sensitive to noise-** It can be sensitive to noisy data, which can affect its convergence.
- **Not suitable for multi-class problems-** It is only suitable for binary classification problems.